

Student Specification: Iterative Anomalous Pattern Initialisation (iK-Means)

Hotels Booking Project - pseudocode, formal specification, and required function signatures

Instructor briefing. Prepared as an original implementation brief derived from the instructor reference implementation and aligned with the standard anomalous-pattern / iK-means formulation.

1. Objective

The task is to implement the iK-means initialisation procedure that extracts anomalous clusters one by one and uses their centroids as the initial seeds for a subsequent k-means run.

Your implementation must reproduce the algorithmic logic of anomalous-pattern extraction, not call a library routine that already performs iK-means.

2. Mathematical setting and notation

- Let $X \in \mathbb{R}^{n \times d}$ be the input matrix with n observations (rows) and d numerical features (columns). Row i is denoted x_i and column j is denoted $X_{:,j}$.
- Let $\mu \in \mathbb{R}^d$ be the grand-mean vector, where $\mu_j = \frac{1}{n} \sum_i X[i, j]$.
- Let $r \in \mathbb{R}^d$ be the feature-scale vector. In this project use range scaling: $r_j = \max_i X[i, j] - \min_i X[i, j]$. If $r_j = 0$, replace it by 1.
- Define the normalised squared distance between two points u and v by

$$\delta_r(u, v) = \sum_{j=1}^d \left(\frac{u_j - v_j}{r_j} \right)^2.$$

- Define the standardised representation $y_i = (x_i - \mu)/r$. In this space the reference point is the origin 0. Computing $\delta_r(x_i, \mu)$ in the original space is equivalent to computing $\|y_i\|_2^2$ in the standardised space.
- Define the total normalised data scatter by

$$D = \sum_{i=1}^n \|y_i\|_2^2.$$

3. Required inputs and outputs

Role	Symbol / name	Specification
Input	X	Numeric matrix of shape (n, d) . The implementation assumes that missing values have already been handled before the function is called.
Input	min_cluster_size τ	Positive integer threshold used to discard anomalous clusters that are too small to serve as reliable k-means seeds.
Input	tol = ε	Strictly positive convergence tolerance for centroid updates. Stop when the centroid change is at most ε , or when cluster membership no longer changes.
Input	max_iter	Maximum number of AP refinement iterations for one anomalous cluster. This is a numerical safeguard.
Output	ap_clusters	List of anomalous-cluster records. Each record should store the row indices in the cluster, the raw centroid, the standardised centroid, the cluster size, and the percentage contribution to total scatter.
Output	init_centroids	Matrix of shape (k, d) containing the standardised centroids of the retained anomalous clusters, where k is the number of clusters whose size is at least τ .

4. Formal algorithmic specification

1. Compute the grand mean μ and the feature scales r using range scaling, replacing any zero range by 1.
2. Compute the total normalised scatter $D = \sum_i \|(x_i - \mu)/r\|_2^2$.
3. Initialise **remains** = $[0, 1, \dots, n - 1]$ and an empty list **ap_clusters**.
4. While **remains** is not empty, select as tentative seed the remaining entity farthest from μ under the normalised squared distance δ_r .
5. From this seed, build one anomalous cluster by alternating two operations: (a) assign to the cluster every remaining point that is strictly closer to the tentative centroid than to μ ; (b) replace the tentative centroid by the component-wise mean of the currently assigned cluster.
6. Repeat the previous assignment/update cycle until the membership set stops changing or the centroid movement is at most ε .
7. Store the extracted cluster together with: (i) its raw centroid c , (ii) its standardised centroid $z = (c - \mu)/r$, and (iii) its percentage contribution to the total scatter, namely

- $100 \times |S| \times \|z\|_2^2 / D$ (take this value as 0 if $D = 0$).
8. Remove the cluster indices from **remains** and repeat the extraction process on the residual set.
 9. After all anomalous clusters have been extracted, retain only those with size at least τ .
 10. Return the retained standardised centroids as the iK-means initialisation for the downstream k-means stage.

5. Pseudocode

Algorithm 1 Main iK-means initialisation procedure

Input: X ($n \times d$ numeric data matrix), τ (minimum cluster size), ε (convergence tolerance), max_iter

Output: **ap_clusters**, **init_centroids**

```

1: Compute  $\mu_j = \text{mean}_i X[i, j]$  for  $j = 1, \dots, d$ .
2: Compute  $r_j = \max_i X[i, j] - \min_i X[i, j]$  for  $j = 1, \dots, d$ .
3: If  $r_j = 0$ , set  $r_j \leftarrow 1$ .
4: Compute  $Y[i, j] = (X[i, j] - \mu_j) / r_j$ .
5: Compute  $D = \sum_i \|Y_i\|^2$ .
6: remains  $\leftarrow [0, 1, \dots, n - 1]$ 
7: ap_clusters  $\leftarrow []$ 
8: while remains is not empty do
9:    $q \leftarrow \arg \max_{i \in \text{remains}} \delta_r(x_i, \mu)$ 
10:  seed  $\leftarrow X[q, :]$ 
11:   $(S, c) \leftarrow \text{EXTRACT-ANOMALOUS-CLUSTER}(X, \text{remains}, r, \mu, \text{seed}, q, \varepsilon, \text{max\_iter})$ 
12:   $z \leftarrow (c - \mu) / r$ 
13:  if  $D > 0$  then
14:    scatter_pct  $\leftarrow 100 \times |S| \times \|z\|^2 / D$ 
15:  else
16:    scatter_pct  $\leftarrow 0$ 
17:  end if
18:  Append record(indices =  $S$ , centroid_raw =  $c$ , centroid_std =  $z$ , size =  $|S|$ , scatter_pct = scatter_pct) to ap_clusters
19:  remains  $\leftarrow \text{remains} \setminus S$ 
20: end while
21: retained  $\leftarrow$  all records in ap_clusters with size  $\geq \tau$ 
22: if retained is empty then
23:   Raise ValueError("No anomalous cluster satisfies the minimum size.")
24: end if
25: init_centroids  $\leftarrow$  stack(record.centroid_std for each record in retained)
26: return ap_clusters, init_centroids

```

Algorithm 2 Extraction of one anomalous cluster**Input:** X , remains , r , μ , seed , seed_index , ε , max_iter **Output:** S (cluster indices), c (raw centroid of S)

```

1:  $c \leftarrow \text{seed}$ 
2:  $S_{\text{prev}} \leftarrow \emptyset$ 
3: for  $t = 1$  to  $\text{max\_iter}$  do
4:    $S \leftarrow \{i \in \text{remains} : \delta_r(x_i, c) < \delta_r(x_i, \mu)\}$ 
       $\triangleright$  Strict inequality is required. Ties stay with the reference point  $\mu$ .
5:   if  $S$  is empty then
6:      $S \leftarrow \{\text{seed\_index}\}$   $\triangleright$  Numerical safeguard
7:   end if
8:    $c_{\text{new}} \leftarrow$  component-wise mean of  $\{x_i : i \in S\}$ 
9:   if  $S = S_{\text{prev}}$  then
10:    return  $S, c_{\text{new}}$ 
11:   end if
12:   if  $\|c_{\text{new}} - c\|_2 \leq \varepsilon$  then
13:    return  $S, c_{\text{new}}$ 
14:   end if
15:    $c \leftarrow c_{\text{new}}$ 
16:    $S_{\text{prev}} \leftarrow S$ 
17: end for
18: return  $S, c$ 

```

Algorithm 3 Auxiliary definitions

```

1: function CENTROID( $X, S$ )
2:   Return the component-wise mean of the rows of  $X$  indexed by  $S$ 
3: end function

4: function NORMALIZED-SQUARED-DISTANCES( $X, I, r, p$ )
5:   For each  $i \in I$ , return  $\delta_r(x_i, p)$ 
6: end function

```

6. Required Python function signatures

```

from dataclasses import dataclass
from numpy.typing import NDArray
import numpy as np

FloatArray = NDArray[np.float64]

@dataclass(frozen=True)
class APCluster:
    indices: list[int]
    centroid_raw: FloatArray
    centroid_std: FloatArray
    size: int
    scatter_pct: float

```

```
def compute_feature_statistics(  
    X: FloatArray,  
    use_unit_ranges: bool = False,  
) -> tuple[FloatArray, FloatArray, float]:  
    ...
```

```
def normalized_squared_distances(  
    X: FloatArray,  
    indices: list[int],  
    scales: FloatArray,  
    reference: FloatArray,  
) -> FloatArray:  
    ...
```

```
def cluster_centroid(  
    X: FloatArray,  
    indices: list[int],  
) -> FloatArray:  
    ...
```

```
def separate_cluster(  
    X: FloatArray,  
    indices: list[int],  
    scales: FloatArray,  
    a: FloatArray,  
    b: FloatArray,  
) -> list[int]:  
    ...
```

```
def extract_anomalous_cluster(  
    X: FloatArray,  
    indices: list[int],  
    scales: FloatArray,  
    mean: FloatArray,  
    initial_centroid: FloatArray,  
    seed_index: int,  
    tol: float = 1e-12,  
    max_iter: int = 10_000,  
) -> tuple[list[int], FloatArray]:  
    ...
```

```
def ikmeans_initialize(  
    ...
```

```

X: FloatArray,
min_cluster_size: int,
tol: float = 1e-12,
max_iter: int = 10_000,
use_unit_ranges: bool = False,
) -> tuple[list[APCluster], FloatArray]:
    ...

```

7. Implementation conventions and corner cases

- Distances must be computed with squared Euclidean distance after division by the feature scales r .
- The assignment rule must use a strict comparison: assign row i to the anomalous cluster only if $\delta_r(x_i, c) < \delta_r(x_i, \mu)$. Ties remain on the reference side.
- Cluster indices must always refer to row numbers of the original matrix X . For deterministic behaviour, return them in increasing order.
- Perform the core numerical calculations in `float64`, even if the input was read in another numeric type.
- The AP extraction stage must be written explicitly. Do not replace it by calling `sklearn.cluster.KMeans` with $K = 2$ inside the loop.
- A cluster is retained as a seed for downstream k-means only if its size is at least `min_cluster_size`.
- If no extracted cluster satisfies the size threshold, the function must raise a descriptive `ValueError`.

8. What will be assessed

- correct computation of μ , r , standardised data, and total scatter D
- correct choice of the farthest remaining entity as the initial AP seed
- correct alternating assignment/update cycle for one anomalous cluster
- correct removal of extracted clusters from the residual set
- correct filtering by the minimum cluster size threshold
- correct construction of the initial centroid matrix returned by the algorithm

9. References

- Mirkin, B. (2005). *Clustering for Data Mining: A Data Recovery Approach*. Chapman & Hall/CRC.
- Chiang, M. M.-T., & Mirkin, B. (2010). Intelligent Choice of the Number of Clusters in K-Means Clustering: An Experimental Study with Different Cluster Spreads. *Journal of Classification*, 27(1), 3-40.